

CHI2401: Software Architecture

Models Report

Nguyen The Toan
2560895

Deadline: Mar. 24, 2025 @ 12:00 GMT
Feedback: May. 05, 2025

2000 $\pm$ 10%

Table of Contents

List of Figures	3
1 Introduction	4
2 Domain Analysis and Model	5
2.1 Domain Background.....	5
2.1.1 High-Energy Physics	5
2.1.2 Large Hadron Collider	5
2.1.3 ATLAS	6
2.1.4 Event Filter Tracking and Data Acquisition	6
2.1.5 A Common Tracking Software (ACTS).....	7
2.2 Domain Model	8
2.3 Utility Tree	8
2.4 Use Case Diagram	9
3 Models	10
3.1 C4	10
3.2 4+1	12
3.3 Architectural Style.....	13
4 Critical Comparative Analysis.....	14
5 References	15
6 Appendix A:	18
7 Appendix n:... ..	19

List of Figures

Figure 1: A Visualisation of Particle Collision Event (The Accelerator Complex CERN, n.d.)..	5
Figure 2: Large Hadron Collider (CERN, n.d.)	5
Figure 3: A Toroidal LHC Apparatus (Detector & Technology, n.d.)	6
Figure 4: Event Filter Tracking and Data Acquisition (Ai et al., 2022)	7
Figure 5: A Common Tracking Software (ACTS) (Ai et al., 2022)	7
Figure 6: ATLAS Event Filter Tracking and Data Acquisition Domain Model	8
Figure 7: Utility Tree for ATLAS Software Qualities	8
Figure 8: Use Case Diagram	9
Figure 9: C4: Context.....	10
Figure 10: C4: Containers.....	10
Figure 11: C4: Components.....	11
Figure 12: C4: Code	11
Figure 13: 4+1 View of ACTS	12
Figure 14: Plug-in components in ACTS (Ai et al., 2022)	13

1 Introduction

The Large Hadron Collider at CERN is the highest-energy proton collider ever built and is instrumented by four major experiments: CMS, ATLAS, LHCb and ALICE. These detectors operate in an extremely data-intensive environment.

However, not all recorded signals can be permanently stored. Therefore, the data acquisition chain must filter events in real time and retain only the most physically relevant information for later analysis and broader physics studies.

In this context, tracking software becomes a critical component of event reconstruction, because it transforms raw detector responses into trajectories of charged particles. However, as emphasized in A Common Tracking Software Project, tracking performance is strongly coupled to detector geometry and experimental conditions, which has historically led each collaboration to develop its own software stack (Ai et al., 2022).

To address this fragmentation, A Common Tracking Software (ACTS) was created as an open-source, experiment-independent framework that can be shared across detectors. Its main objectives are to preserve and extend mature tracking algorithms, provide a research and development platform for new methods and heterogeneous computing, and offer a reusable foundation for future tracking detectors.

ACTS is therefore designed around minimal external dependencies, abstraction of detector-specific geometry and event data, and thread-safe, configurable components that support scalable reconstruction workflows.

2 Domain Analysis and Model

2.1 Domain Background

2.1.1 High-Energy Physics

High-Energy Physics is a branch of particle physics that investigates fundamental particles and their behaviour at extremely high energies using particle accelerators. These studies aim to understand matter, forces, and conditions similar to the early universe (Butler et al., 2022). Figure 1 below visualises a particle collision event.

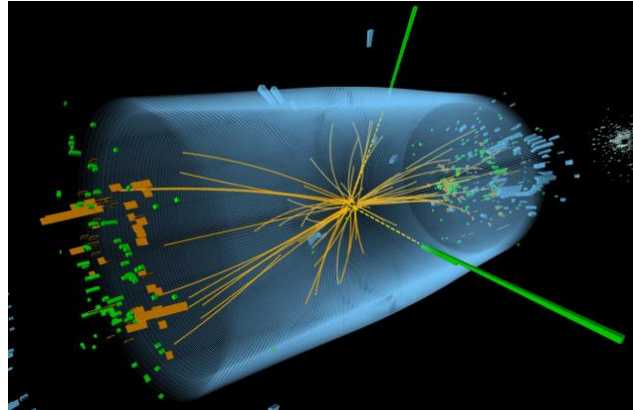


Figure 1: A Visualisation of Particle Collision Event (The Accelerator Complex | CERN, n.d.)

2.1.2 Large Hadron Collider

Within high-energy physics, The Large Hadron Collider is regarded as the most powerful particle accelerator, designed to facilitate proton collisions at extremely high energies in order to investigate fundamental particles and force. It enables precision tests of the Standard Model and investigations into physics beyond current theoretical limits (Bass et al., 2021). Figure 2 provides an overview of the CERN accelerator complex, highlighting the chain of accelerators leading to the LHC.

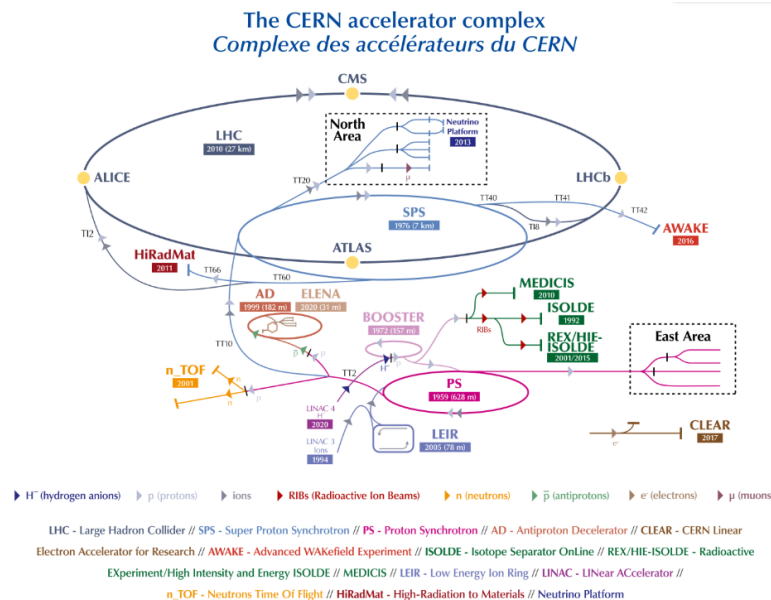


Figure 2: Large Hadron Collider (CERN, n.d.)

2.1.3 ATLAS

ATLAS detector is a multi-purpose detector operating at the Large Hadron Collider, developed for experiments in high-energy physics. It analyses proton–proton collision data to study fundamental particles and to search for physics beyond the Standard Model through precision measurements and rare event detection (Aad et al., 2022). The overall architecture of the ATLAS detector is illustrated in Figure 3, highlighting its layered structure and the functional roles of its major subsystems.

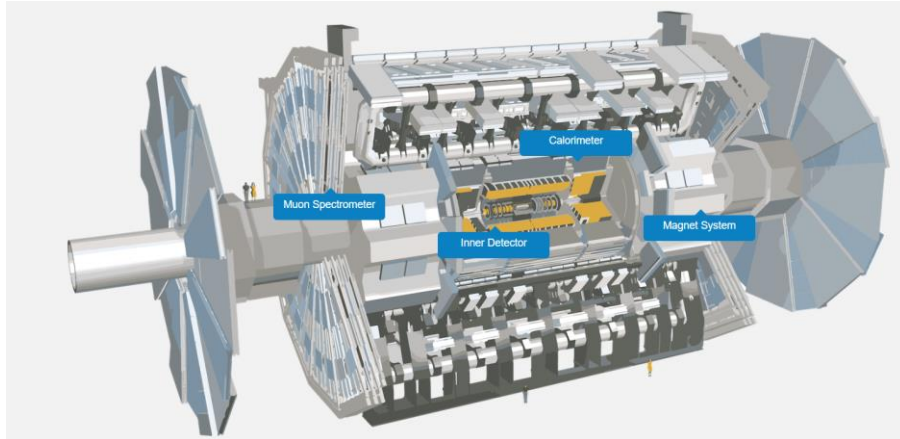


Figure 3: A Toroidal LHC Apparatus (Detector & Technology, *n.d.*)

2.1.4 Event Filter Tracking and Data Acquisition

In high-energy physics experiments, the Event Filter constitutes the ultimate stage of the software trigger system responsible for selecting physically significant collision events from the raw detector data stream. Within this stage, track reconstruction algorithms are deployed online to discriminate signal events from background, necessitating computationally efficient and highly parallelisable tracking implementations (Ai et al., 2022; Osborn et al., 2021). Figure 4 presents the architecture of the Data Acquisition system and Event Filter Tracking, outlining the flow of data from detector inputs to final storage.

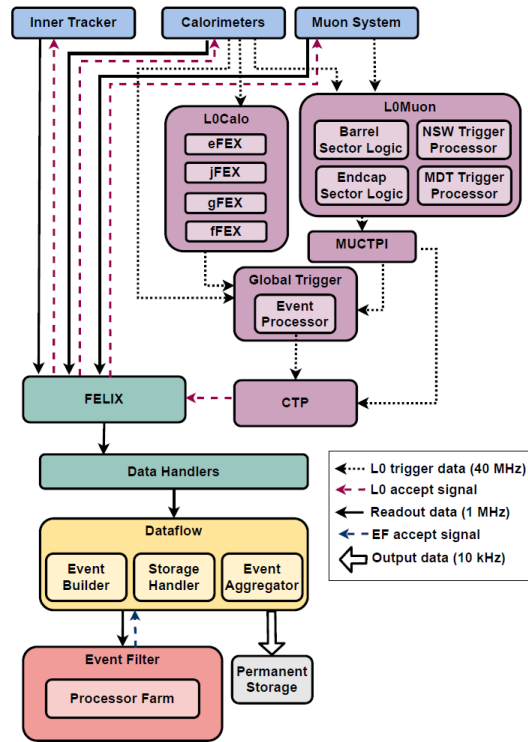


Figure 4: Event Filter Tracking and Data Acquisition (Ai et al., 2022)

2.1.5 A Common Tracking Software (ACTS)

ACTS constitutes an experiment-independent, open-source framework engineered for reconstructing charged particle tracks in high-energy physics environment. It provides modular architecture, thread-safe algorithms encompassing geometry modelling, track propagation, seeding, and Kalman-based fitting, deliberately decoupled from any specific detector configuration or experimental framework to maximise reusability across diverse collider experiments (Ai et al., 2022). Figure 5 illustrates the data processing workflow in the ACTS framework, from detector measurements to reconstructed vertices and tracks for analysis.

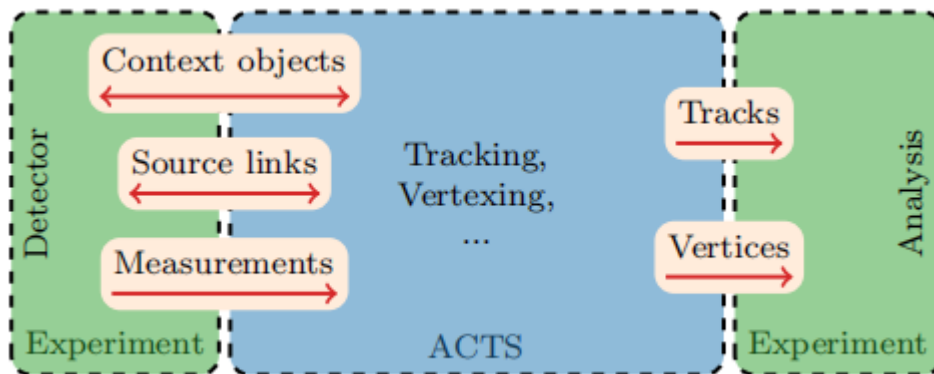


Figure 5: A Common Tracking Software (ACTS) (Ai et al., 2022)

2.2 Domain Model

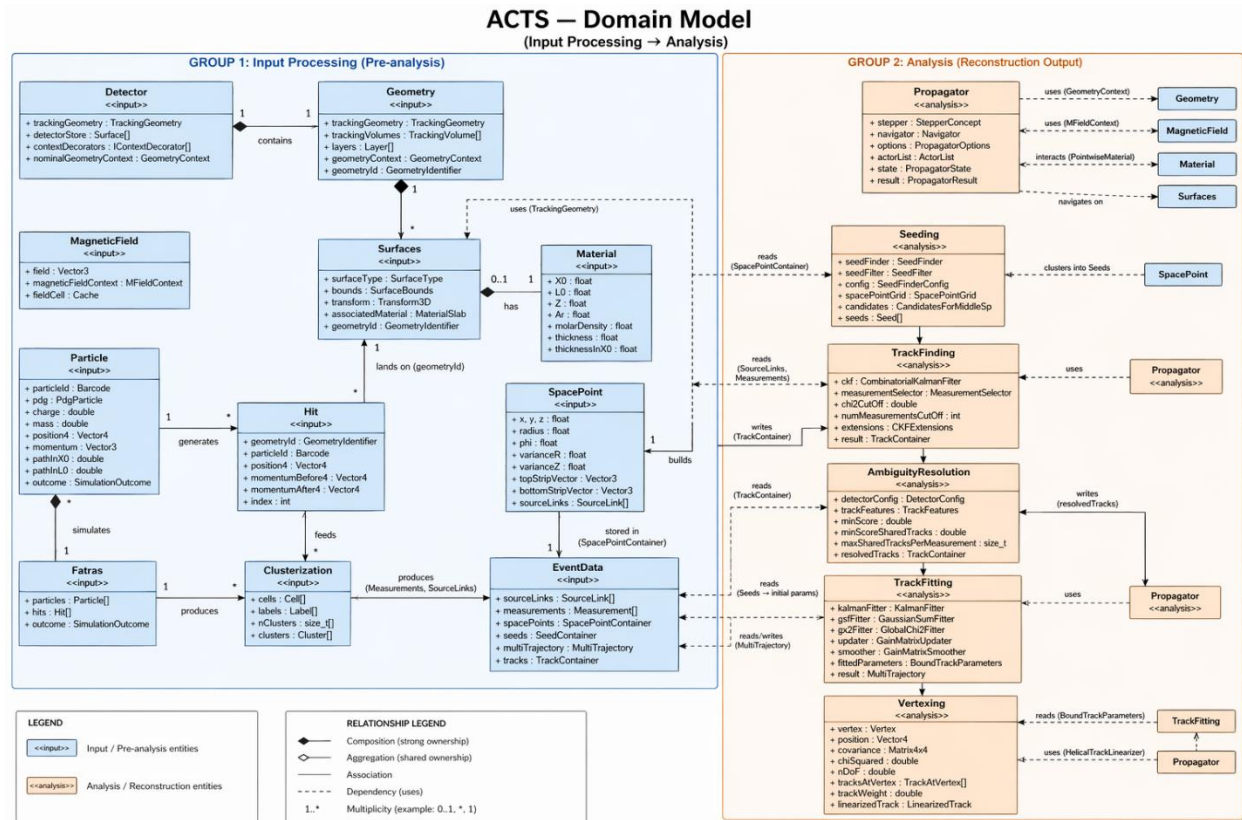


Figure 6: ATLAS Event Filter Tracking and Data Acquisition Domain Model

2.3 Utility Tree

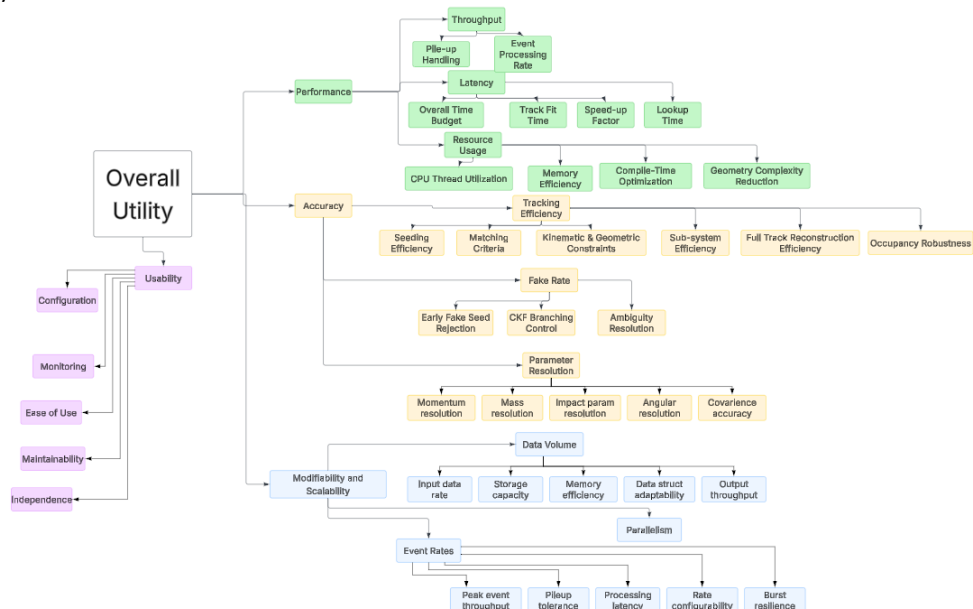


Figure 7: Utility Tree for ATLAS Software Qualities

2.4 Use Case Diagram

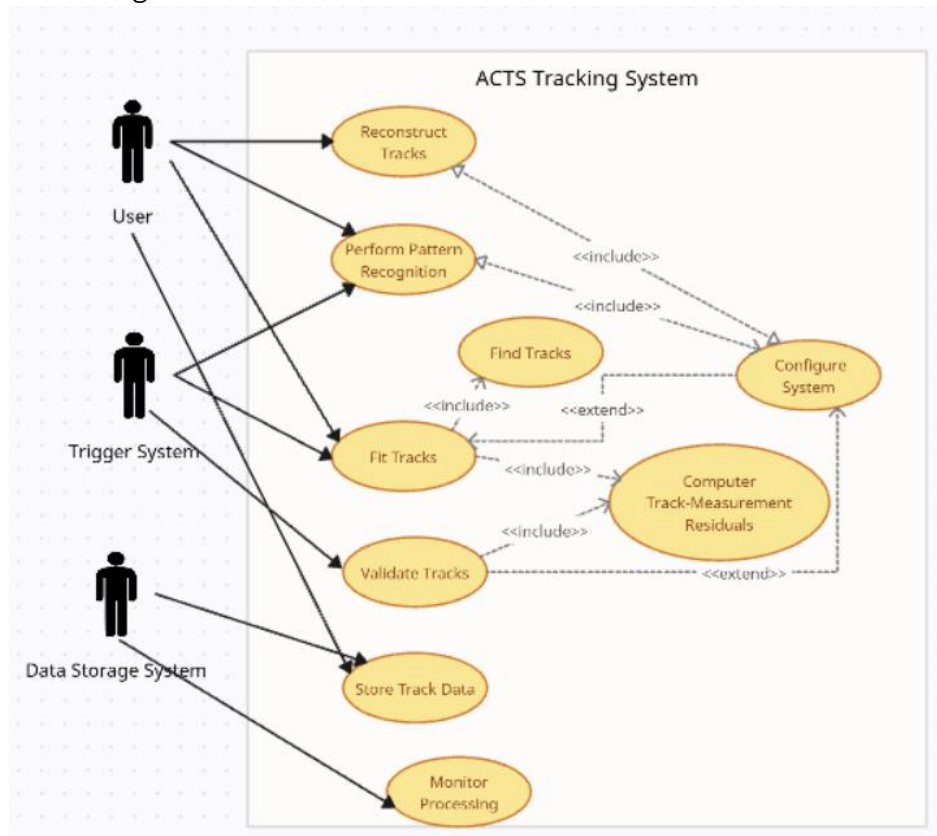


Figure 8: Use Case Diagram

3 Models

3.1 C4

Figure 9 illustrates the C4 Context view of the ACTS system. The ACTS System centralises experiment-independent reconstruction, thereby isolating the framework from external dependencies. Users represent physicists who interact with ACTS through external orchestration frameworks. The Detector is modelled as an external entity, as ACTS consumes but does not own geometry descriptions. Data Storage and Analysis are treated as separate components to preserve ACTS's statelessness with respect to output persistence.

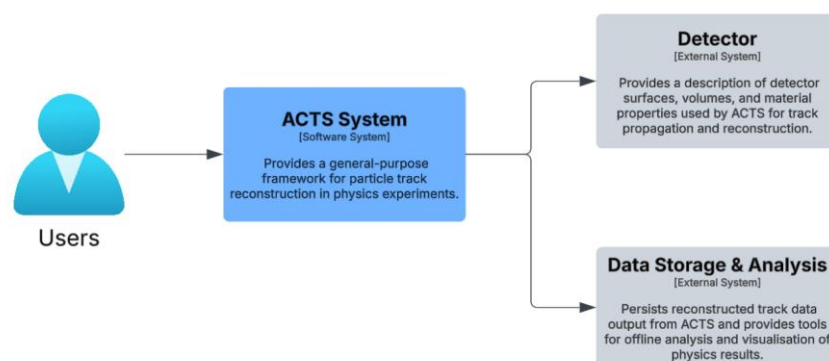


Figure 9: C4: Context

Figure 10 presents the C4 Container view of the ACTS system. Data Inputs enforce the experiment-independence boundary by normalising raw signals into ACTS compatible representations. The Tracking Engine is isolated to encapsulate the stateless algorithmic core responsible for executing a thread-safe reconstruction pipeline. The Geometry Service is decoupled to separate geometric state management from computational tracking logic. The Output component is segregated to decouple reconstruction processes from downstream serialisation and persistence concerns.

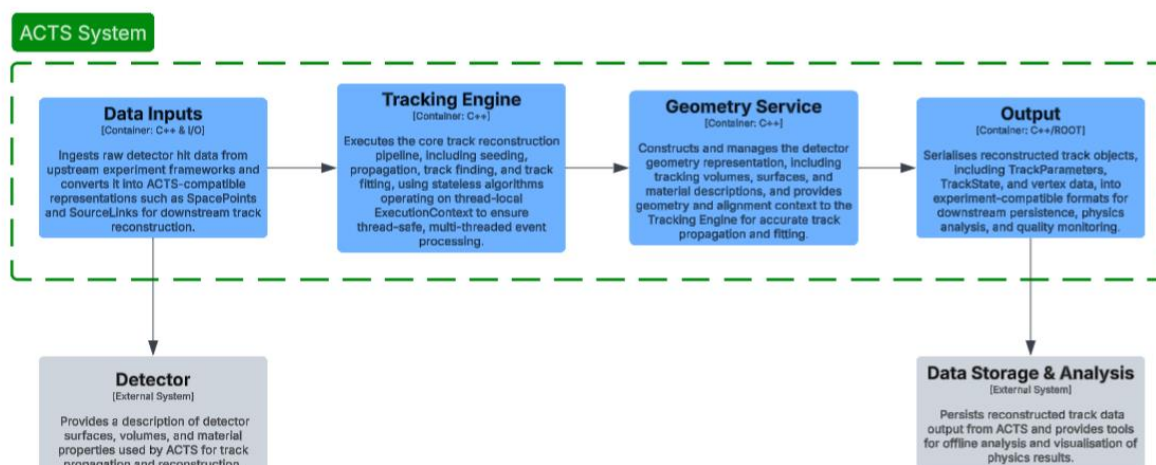


Figure 10: C4: Containers

Figure 11 presents the C4 Component view of the ACTS system. Hit Processing is separated to encapsulate the refinement of raw detector signals prior to seeding. Seed

Finding is isolated to represent the initial generation of track hypotheses under geometric and kinematic constraints. Track Finding is distinguished to encapsulate pattern recognition processes that extend seeds through detector measurements. Track Fitting is separated to isolate statistical state estimation, refining track candidates into final parameters. The Magnetic Field Service is decoupled as a shared infrastructure component that provides field values to all propagation-related processes.

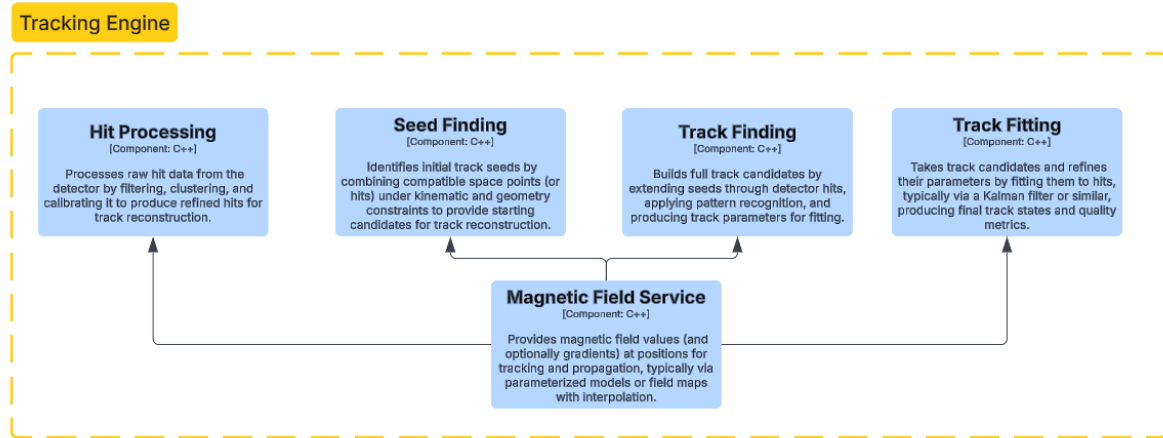


Figure 11: C4: Components

Figure 12 presents the C4 Code view of the ACTS system. The Kalman Filter represents the primary iterative Bayesian estimator used for sequential track state refinement. The Global Chi-Square Fitter provides an alternative global optimisation approach that minimises the overall residual chi-square. Material Effects encapsulate the necessary physics corrections for energy loss and multiple Coulomb scattering. Track State is defined as an immutable data structure that persists position, momentum, and covariance at each detector surface.

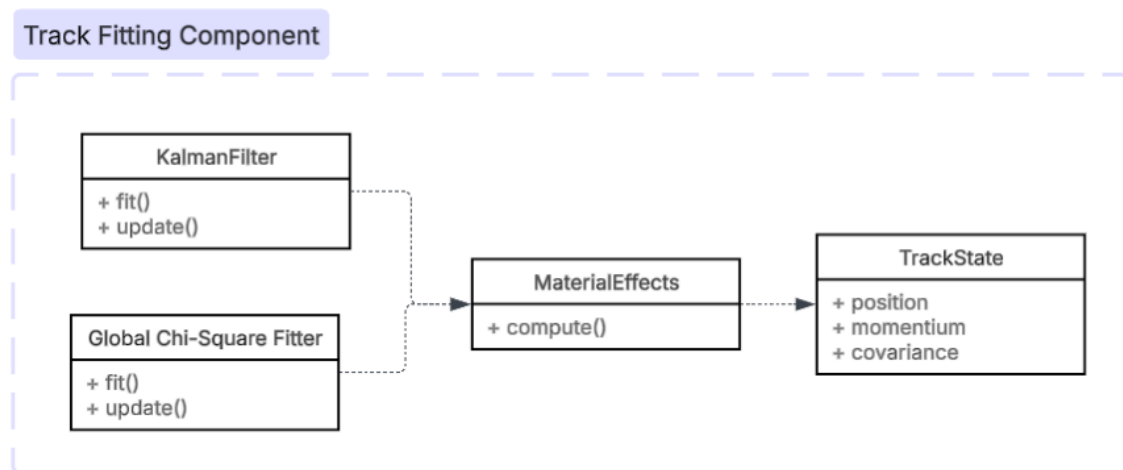


Figure 12: C4: Code

3.2 4+1

Figure 13 presents the 4+1 model of the ACTS system, capturing its multi-dimensional architectural concerns in a comprehensive manner. The Logical View centres on Track, Hit, Surface, and Trajectory Element because these represent the core domain abstractions around which all reconstruction logic is organised. The Process View depicts parallel event streams processed by independent ACTS instances, reflecting the framework's stateless, thread-safe algorithm design. The Development View mirrors the actual source tree structure, exposing module boundaries that enforce separation of concerns. The Physical View, referenced from the ACTS GitHub repository, maps CPU nodes, GPU nodes, storage, and detectors within the CERN computing cluster, accurately representing heterogeneous deployment reality. The Scenario View anchors all perspectives to the primary use case of particle track reconstruction, providing functional coherence across views.

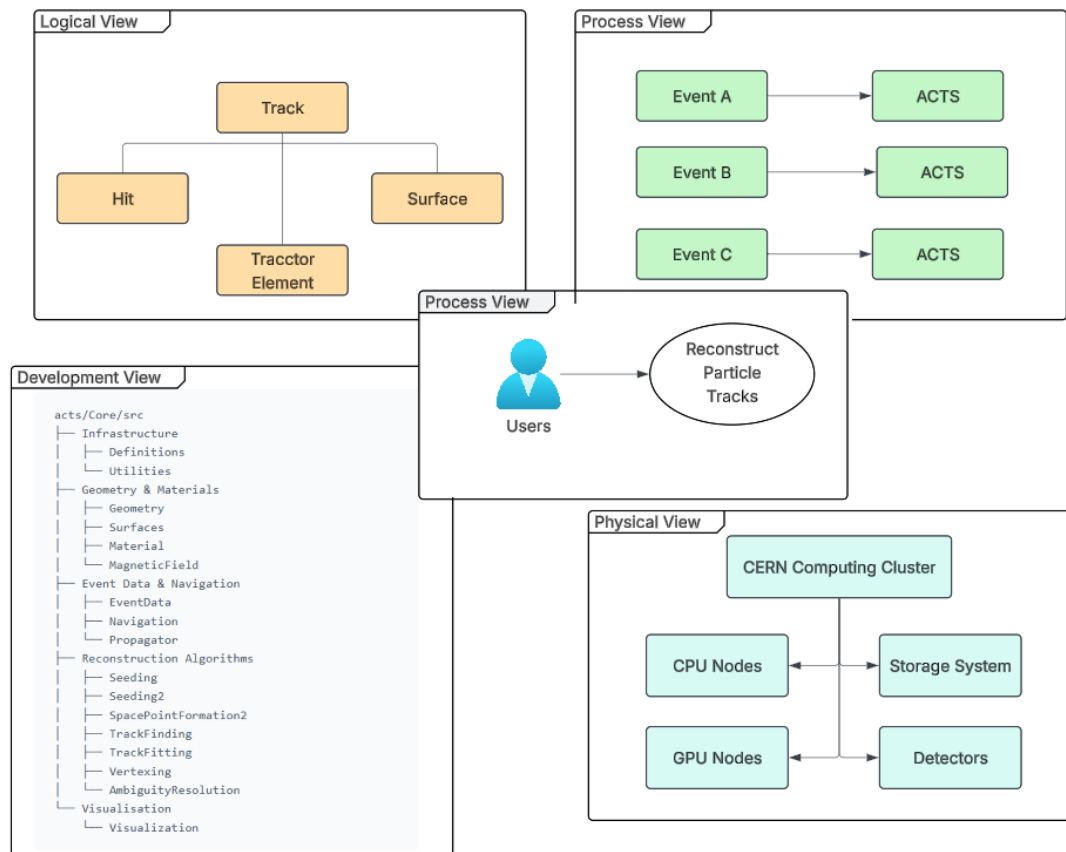


Figure 13: 4+1 View of ACTS

3.3 Architectural Style

The most appropriate architectural style for ACTS is Microkernel architecture. This style calls for a lean, stable core extended through discrete plugin components, precisely what ACTS implements: the core library provides essential reconstruction algorithms with deliberately minimal external dependencies, while a surrounding plugin layer handles experiment-specific extensions (Ai et al., 2022). The decoupling keeps the core stable and independently verifiable whilst accommodating the technological diversity of a community spanning ATLAS, sPHENIX, and FASER (Varni, 2025). Complementary Layered and Stateless Concurrent patterns reinforce this primary style, governing source-level dependency boundaries and thread-safe parallel execution respectively (Ai et al., 2022). Figure 14 presents a visual representation of the plug-in components in ACTS.

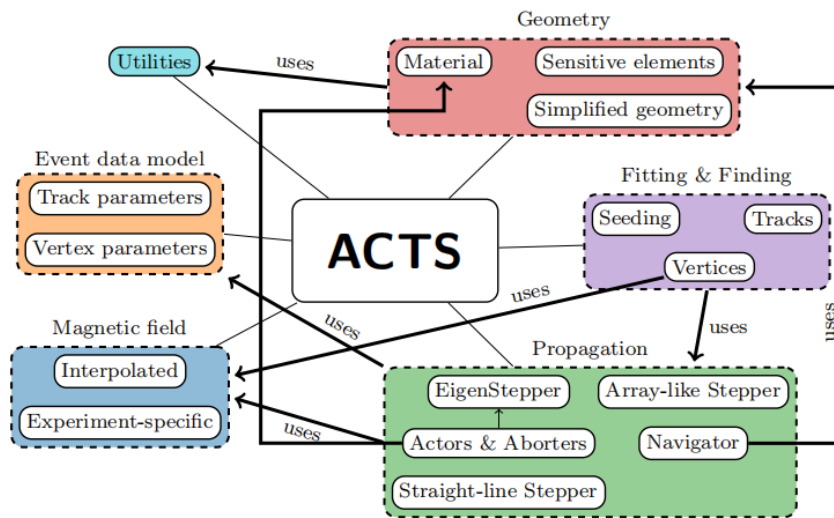


Figure 14: Plug-in components in ACTS (Ai et al., 2022)

4 Critical Comparative Analysis

Each modelling technique addresses a distinct dimension of software architecture, yet none covers the full picture alone.

The Use Case Diagram maps system functionality from an actor's perspective, capturing what the system must do and for whom or put differently, it surfaces the goal-oriented interactions between external parties and the system boundary (van Vliet, 2007). This makes it the most accessible artefact for non-technical stakeholders. However, it treats non-functional requirements as an afterthought, handling them in supplementary documentation rather than integrating them into the model itself (Glinz, 2007). Compared to the Domain Model, which operates entirely within the problem space, Use Case Diagrams sit at the functional boundary between user intent and system response.

The Domain Model maps the conceptual entities and their relationships within a problem domain. Stated otherwise, it builds a shared vocabulary between developers and domain experts (Gharbi et al., 2019). Unlike the Use Case Diagram, it says nothing about system behaviour or quality. Its principal weakness is subjectivity: domain boundaries shift over time, and different analysts may produce incompatible models (van Vliet, 2007).

The Utility Tree decomposes quality attributes hierarchically into measurable scenarios, giving stakeholders a structured basis for prioritising NFRs (Gharbi et al., 2019). Where the Domain Model concerns itself with what exists and the Use Case Diagram with what the system does, the Utility Tree asks how well. Its limitation is scalability: large systems generate more scenarios than any assessment can practically evaluate (van Vliet, 2007).

The C4 Model organises architecture across four levels of abstraction, moving from system context down to code. Alternatively, it progressively narrows the lens from stakeholder-facing overview to developer-facing detail. Unlike the 4+1 Model, C4 is deliberately lightweight and does not mandate separate viewpoints per concern. However, its Code level is rarely maintained in practice (Peacock, 2023).

The 4+1 Model separates architectural concerns across five distinct views, each targeting a different stakeholder group. In other words, it distributes analysis across logical, process, development, physical, and scenario dimensions simultaneously. This is its clear advantage over C4, which collapses those concerns into a single abstraction hierarchy. The trade-off is overhead: for smaller systems, the logical and development views often duplicate one another, making the full model unnecessarily burdensome (Kruchten, 1995).

In conclusion, Use Case Diagrams and the Utility Tree complement each other at the requirements level, the Domain Model grounds the problem space and C4 versus 4+1 reflects a choice between communicative simplicity and analytical completeness.

5 References

- Aad, G., Abbott, B., Abbott, D. C., Abeling, K., Abidi, S. H., Aboulhorma, A., Abramowicz, H., Abreu, H., Abulaiti, Y., Abusleme Hoffman, A. C., Acharya, B. S., Achkar, B., Adam, L., Bourdarios, C. A., Adamczyk, L., Adamek, L., Addepalli, S. V., Adelman, J., Adiguzel, A., ... The ATLAS Collaboration. (2022). A detailed map of Higgs boson interactions by the ATLAS experiment ten years after the discovery. *Nature*, 607(7917), 52–59.
<https://doi.org/10.1038/s41586-022-04893-w>
- Ai, X., Allaire, C., Calace, N., Czirkos, A., Elsing, M., Ene, I., Farkas, R., Gagnon, L.-G., Garg, R., Gessinger, P., Grasland, H., Gray, H. M., Gumpert, C., Hrdinka, J., Huth, B., Kiehn, M., Klimpel, F., Kolbinger, B., Krasznahorkay, A., ... Zhang, J. (2022). A Common Tracking Software Project. *Computing and Software for Big Science*, 6(1), 8.
<https://doi.org/10.1007/s41781-021-00078-8>
- Bass, S. D., De Roeck, A., & Kado, M. (2021). The Higgs boson implications and prospects for future discoveries. *Nature Reviews Physics*, 3(9), 608–624.
<https://doi.org/10.1038/s42254-021-00341-2>
- Butler, J., Chivukula, R. S., de Gouvea, A., Han, T., Kim, Y.-K., & Cushman, P. (2022). How high-energy physics plans its future. *Nature Reviews Physics*, 4(9), 565–567.
<https://doi.org/10.1038/s42254-022-00493-9>
- Detector & Technology*. (n.d.). Retrieved 4 May 2026, from
<https://atlas.cern/Discover/Detector>
- Gharbi, M., Koschel, A., & Rausch, A. (2019). *Software Architecture Fundamentals: A Study Guide for the Certified Professional for Software Architecture® - Foundation Level - ISAQB Compliant*. dpunkt.verlag.
<http://ebookcentral.proquest.com/lib/hud/detail.action?docID=5722381>

Glinz, M. (2007). On Non-Functional Requirements. *15th IEEE International Requirements Engineering Conference (RE 2007)*, 21–26. <https://doi.org/10.1109/RE.2007.45>

Kruchten, P. B. (1995). The 4+1 View Model of architecture. *IEEE Software*, 12(6), 42–50. <https://doi.org/10.1109/52.469759>

Osborn, J. D., Frawley, A. D., Huang, J., Lee, S., Costa, H. P. D., Peters, M., Pinkenburg, C., Roland, C., & Yu, H. (2021). Implementation of ACTS into sPHENIX Track Reconstruction. *Computing and Software for Big Science*, 5(1), 23. <https://doi.org/10.1007/s41781-021-00068-w>

Peacock, A. (2023). *Creating Software with Modern Diagramming Techniques*. Pragmatic Programmers, LLC.

The accelerator complex / CERN. (n.d.). Retrieved 25 April 2026, from <https://home.cern/science/accelerators/accelerator-complex>

van Vliet, H. (2007). *Software engineering: Principles and practice* (3rd edn). John Wiley & Sons.

Varni, C. (2025). Integration of the ACTS track reconstruction toolkit in the ATLAS software for HL-LHC operations. *EPJ Web of Conferences*, 337, 01346. <https://doi.org/10.1051/epjconf/202533701346>

Appendices

6 Appendix A: ...

7 Appendix n:...